DOCUMENTACIÓN TÉCNICA.

Examen Técnico Junior Frontend

Que presenta:

Carlos Daniel Quintal Pech

Ingeniero en Sistemas Computacionales

ÍNDICE

Descripción General 3

Tecnologías Utilizadas 3

Arquitectura 4

1. Directorio Raíz 5

2. Directorio src/ 8

3. Relación y Flujo de Datos 11

4. Configuración y Herramientas 12

5.Componentes de la Interfaz de Usuario 13

Buenas Prácticas de Desarrollo Frontend 20

Experiencia de Usuario (UX) 25

Descripción General

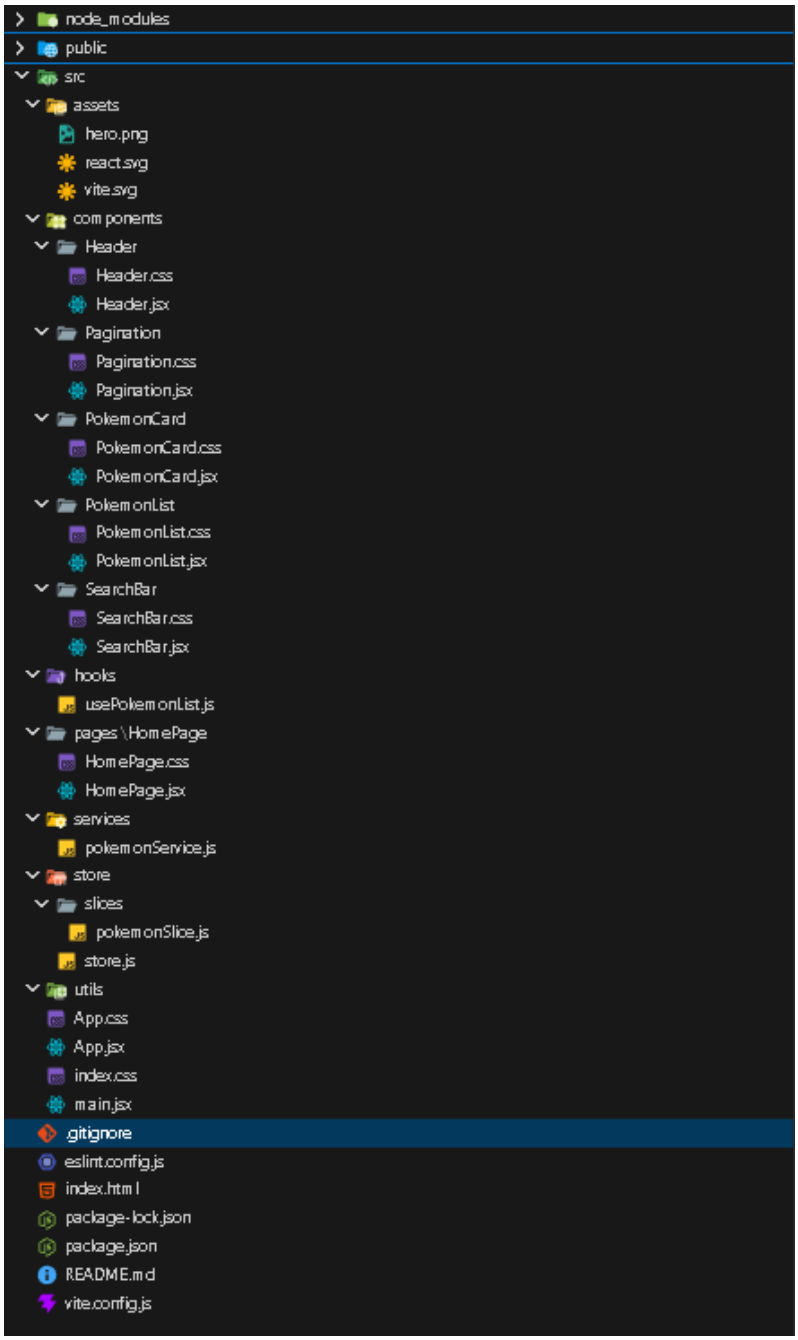
Aplicación web desarrollada con *React* que consume la *PokéAPI* para mostrar una lista de Pokémon con paginación (6 por página), búsqueda por nombre y filtrado por tipo. La interfaz sigue una estética retro inspirada en las consolas Game Boy y la Pokédex clásica, con énfasis en la experiencia de usuario y el diseño responsivo.

Tecnologías Utilizadas

Tecnología	**Versión**	**Propósito**
React	18.2	Librería principal para la construcción de la interfaz de usuario
Vite	4.x	Herramienta de build y servidor de desarrollo rápido
Redux Toolkit	1.9	Manejo del estado global y acciones asíncronas
React Redux	8.1	Conector entre React y Redux
Axios	1.4	Cliente HTTP para consumir la PokéAPI
React Icons	4.10	Iconos (Pokebola, etc.)
NodeJS	v22.22.1	Permite ejecutar todas las herramientas modernas de desarrollo frontend que están escritas en JavaScript.
CSS3	-	Estilizado con enfoque modular y responsive
PokéAPI	-	Fuente de datos de Pokémon

Arquitectura

El proyecto está desarrollado con React y Vite, siguiendo una arquitectura modular que separa responsabilidades en capas: componentes de presentación, lógica de negocio, servicios de API, estado global y utilidades. A continuación, se describe cada elemento de la estructura de archivos y directorios.



1. Directorio Raíz

- ***node_modules/***

  Contiene todas las dependencias instaladas mediante npm. Es generado automáticamente al ejecutar npm install y no debe ser modificado manualmente. Su contenido se define en package.json y package-lock.json.

- ***public/***

  Almacena archivos estáticos que se copiarán directamente en la carpeta de distribución (dist) sin ser procesados por Vite. Normalmente se colocan aquí el favicon.ico, robots.txt, imágenes que no requieren optimización, etc. En el proyecto actual, esta carpeta aparece vacía.

- ***src/***

  Directorio principal que contiene todo el código fuente de la aplicación.

- ***.gitignore***

  Archivo de configuración para Git que especifica qué archivos o carpetas deben ser ignorados en el control de versiones. Por lo general incluye node_modules, dist, archivos de entorno (.env), entre otros.

- ***eslint.config.js***

  Configuración de ESLint, herramienta de linting que analiza el código JavaScript/JSX en busca de errores y malas prácticas. Define reglas, entornos y plugins (por ejemplo, para React) que garantizan la consistencia y calidad del código.

- ***index.html***

  Punto de entrada HTML de la aplicación. Vite lo utiliza como plantilla: en tiempo de compilación, inyecta automáticamente las referencias a los scripts y estilos generados (como main.jsx y los archivos CSS). En este archivo se encuentra el elemento donde se montará la aplicación React.

- ***package.json***

  Manifiesto del proyecto que lista las dependencias (producción y desarrollo), scripts disponibles (como dev, build, preview), metadatos (nombre, versión, descripción) y configuraciones básicas. Es fundamental para la gestión del proyecto con npm.

- ***package-lock.json***

  Archivo generado automáticamente que fija las versiones exactas de todas las dependencias instaladas (incluyendo subdependencias). Garantiza que todos los desarrolladores y entornos de despliegue utilicen las mismas versiones, evitando inconsistencias.

- ***README.md***

  Documentación del proyecto en formato Markdown. Describe cómo instalar, configurar y ejecutar la aplicación, así como cualquier otra información relevante para desarrolladores y usuarios.

- ***vite.config.js***

  Archivo de configuración de Vite. Permite personalizar el comportamiento del bundler: definir plugins (como @vitejs/plugin-react), establecer alias para rutas, configurar el servidor de desarrollo, ajustar opciones de compilación, entre otras tareas.

- ***main.jsx***

  Punto de entrada de React. Se encarga de renderizar el componente raíz (<App />) dentro del elemento root del DOM. Además, envuelve la aplicación con el Provider de Redux para proveer el store global y con React.StrictMode para detectar posibles problemas en desarrollo. Su contenido típico es:

```jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import { Provider } from 'react-redux';
import { store } from './src/store/store';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>
);
```

- ***App.jsx***

  Componente funcional raíz que define la estructura principal de la aplicación. En este proyecto, simplemente importa y renderiza la página HomePage. También puede incluir enrutamiento (React Router) o layouts generales. Actualmente su contenido es:

```jsx
import HomePage from './src/pages/HomePage/HomePage';
import './App.css';

function App() {
  return <HomePage />;
}

export default App;
```

- ***index.css***

  Hoja de estilos global que contiene reglas CSS aplicables a toda la aplicación. Puede incluir resets, variables CSS, estilos base para elementos HTML, etc. Se importa en main.jsx para que afecte a todos los componentes.

2. Directorio src/

Contiene el código fuente organizado por funcionalidad.

2.1 assets/

Almacena recursos estáticos que serán procesados por Vite (optimización, hashing, etc.).

- *hero.png* – Imagen de cabecera o banner principal.
- *react.svg* – Logotipo de React, utilizado posiblemente en componentes o como marcador de posición.
- *vite.svg* – Logotipo de Vite, usado con fines demostrativos o de branding.

2.2 components/

Componentes de interfaz de usuario reutilizables. Cada componente se organiza en su propia carpeta, conteniendo su lógica (.jsx) y estilos (.css), lo que facilita el mantenimiento y la reutilización.

- ***Header/***
    - Header.jsx – Componente que muestra la cabecera de la aplicación (logotipo, menú de navegación, etc.).
    - Header.css – Estilos específicos para el encabezado.
- ***Pagination/***
    - Pagination.jsx – Componente que proporciona controles de paginación (botones "Anterior", "Siguiente" y/o números de página) para navegar a través de listas de elementos.
    - Pagination.css – Estilos para los controles de paginación.

- ***PokemonCard/***
    - PokemonCard.jsx – Tarjeta individual que presenta información resumida de un Pokémon: nombre, imagen, tipos, etc. Se utiliza dentro de listas.
    - PokemonCard.css – Estilos para la tarjeta.
- ***PokemonList/***
    - PokemonList.jsx – Componente que recibe una lista de Pokémon y renderiza múltiples PokemonCard. Puede incluir lógica de visualización (grid, lista) y manejo de estados vacíos o de carga.
    - PokemonList.css – Estilos para el contenedor de la lista.
- ***SearchBar/***
    - SearchBar.jsx – Componente de entrada de texto que permite al usuario filtrar Pokémon por nombre. Emite eventos de cambio o búsqueda al componente padre.
    - SearchBar.css – Estilos para la barra de búsqueda.

2.3 hooks/

Contiene hooks personalizados que encapsulan lógica reutilizable con estado y efectos secundarios.

- usePokemonList.js
  Hook que abstrae la lógica de obtención de la lista de Pokémon. Posiblemente utiliza el servicio pokemonService.js para hacer peticiones a la API, maneja estados de carga, error y paginación, y expone los datos y funciones necesarias a los componentes.

2.4 pages/

Agrupa los componentes que representan vistas completas de la aplicación. Cada página suele combinar varios componentes de la carpeta components y puede tener su propio estado y lógica.

- ***HomePage/***
    - HomePage.jsx – Página principal de la aplicación. Integra el SearchBar, PokemonList y Pagination. Gestiona el estado de la búsqueda y la página actual, y utiliza el hook usePokemonList o el store para obtener los datos.
    - HomePage.css – Estilos específicos para la página de inicio.

2.5 services/

Capa de servicios que se encarga de la comunicación con APIs externas o de operaciones asíncronas.

- ***pokemonService.js***
  Módulo JavaScript que exporta funciones para interactuar con la PokéAPI. Ejemplos de funciones:
    - fetchPokemonList(offset, limit) – Obtiene una lista paginada de Pokémon.
    - fetchPokemonDetails(nameOrId) – Obtiene los detalles de un Pokémon específico.
      Estas funciones utilizan fetch o axios y devuelven promesas. Separar la lógica de red de los componentes facilita el testing y el mantenimiento.

2.6 store/

Implementación del estado global con Redux Toolkit. Contiene la configuración del store y los slices que definen el estado y los reducers.

- ***slices/***

- pokemonSlice.js – Slice de Redux que define:
    - Estado inicial (lista de Pokémon, página actual, filtro de búsqueda, estado de carga, error, etc.).
    - Reducers síncronos (acciones) para modificar el estado (ej. setPokemonList, setPage, setSearchTerm).
    - Acciones asíncronas (creadas con createAsyncThunk) para obtener datos del servicio y actualizar el estado.
    - Exporta el reducer y las acciones para ser utilizados en componentes.

3. Relación y Flujo de Datos

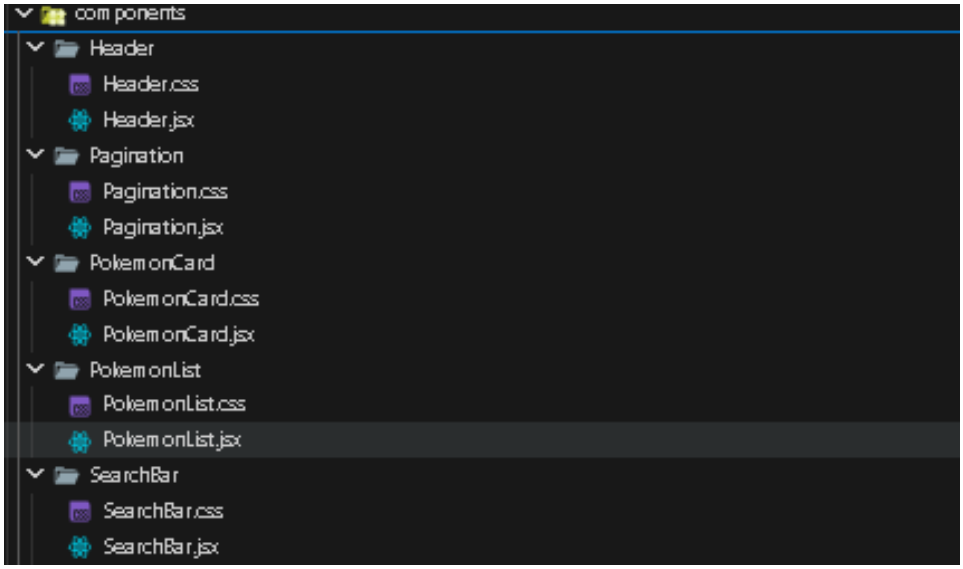
1. **Inicio**: El archivo index.html carga main.jsx.
2. **Renderizado**: main.jsx crea el root de React, envuelve la aplicación con el Provider de Redux y renderiza <App />.
3. **Componente raíz**: App.jsx importa HomePage y la muestra.
4. **Página Home**: HomePage.jsx se suscribe al store de Redux (usando hooks como useSelector y useDispatch) para obtener el estado de los Pokémon y las acciones. También puede utilizar el hook personalizado usePokemonList para manejar la lógica de carga.
5. **Obtención de datos**:
    - Al montarse, HomePage o el hook usePokemonList despacha una acción asíncrona (definida en pokemonSlice.js) que llama a pokemonService.js.
    - El servicio realiza la petición a la PokéAPI y devuelve los datos.
    - El thunk maneja la respuesta y actualiza el store mediante reducers.

6. **Interacción del usuario**:
    - El usuario escribe en SearchBar, lo que dispara una acción para actualizar el término de búsqueda en el store (o un estado local).
    - Al cambiar de página mediante Pagination, se actualiza la página actual y se dispara una nueva carga de datos.
    - PokemonList se suscribe a la lista filtrada del store y renderiza las PokemonCard correspondientes.
7. **Estilos**: Cada componente importa su propio archivo CSS, garantizando que los estilos sean modulares y no contaminen otros componentes. Además, los estilos globales definidos en index.css se aplican en toda la aplicación.

4. Configuración y Herramientas

- **Vite**: Proporciona un servidor de desarrollo rápido y empaquetado optimizado para producción. Su configuración en vite.config.js puede incluir alias para simplificar las importaciones (ej. @ apuntando a src).
- **ESLint**: Mantiene la calidad del código mediante reglas configurables. El archivo eslint.config.js define dichas reglas, adaptadas a React y a las preferencias del equipo.
- **Git**: El archivo .gitignore asegura que los archivos generados y las dependencias no se suban al repositorio.

5.Componentes de la Interfaz de Usuario



5.1 Header

Archivos: Header.jsx, Header.css

Propósito:
Componente de cabecera que muestra el nombre del desarrollador (requisito de la prueba técnica) y un distintivo "Pokédex" con versión. Proporciona identidad visual y se ubica en la parte superior izquierda de la página.

Implementación Técnica:

- Utiliza el icono CgPokemon de la librería react-icons/cg para representar una Pokébola.
- El nombre del desarrollador se muestra en un elemento <h1> con la clase header-name.
- Un badge lateral muestra el texto "Pokédex" y la versión "v1.0" con estilos de borde y sombra para mantener la estética retro.
- Los estilos se definen en Header.css y aplican:
    - Fondo degradado y borde inferior rojo característico.

- Sombra y bordes negros para efecto de relieve.
- Efecto hover sobre el icono que lo hace rotar ligeramente.
- Media queries para adaptación a dispositivos móviles.

Interacción:
El componente es puramente visual y no contiene lógica de negocio ni estado interno.

5.2 Pagination

Archivos: Pagination.jsx, Pagination.css

Propósito:
Proporciona los controles de paginación para navegar entre las páginas de Pokémon, mostrando información del rango actual y permitiendo saltos a la primera, última, anterior y siguiente página, así como a números de página específicos.

Implementación Técnica:

- Conectado al store de Redux mediante useSelector y useDispatch para acceder a currentPage y count (total de Pokémon), y para despachar la acción setPage.
- Calcula totalPages = Math.ceil(count / 6).
- Muestra el rango de Pokémon visibles: (currentPage - 1) * 6 + 1 a min(currentPage * 6, count).
- Genera dinámicamente una lista de números de página a mostrar (hasta 5) usando la función getPageNumbers, que incluye puntos suspensivos para rangos largos.
- Los botones de navegación:
    - "Primera" y "Última" (en texto, sin iconos).

    - Anterior (<) y Siguiente (>) con entidades HTML.
    - Números de página, con la página actual resaltada (clase active).
- Los botones se deshabilitan cuando están en los extremos (primera o última página).
- Al cambiar de página, se ejecuta window.scrollTo({ top: 0, behavior: 'smooth' }) para mejorar la experiencia de usuario.
- El componente se oculta si totalPages <= 1.
- Los estilos en Pagination.css definen un diseño con bordes negros, sombras duras, tipografía 'Press Start 2P' y colores de fondo rojo para botones normales y amarillo para el activo. Incluye media queries para responsive.

5.3 PokemonCard

Archivos: PokemonCard.jsx, PokemonCard.css

Propósito:
Representa una tarjeta individual de Pokémon con información detallada: imagen, nombre, ID, tipos, estadísticas base (HP, ataque, defensa, velocidad), experiencia base y habilidad principal. El diseño se adapta al tipo del Pokémon mediante colores de fondo y bordes.

Implementación Técnica:

- Recibe como prop name (nombre del Pokémon).
- Utiliza estado local (useState) para almacenar los datos del Pokémon (pokemon) y el estado de carga (loading).
- En useEffect, llama a fetchPokemonDetail(name) (servicio) para obtener los detalles completos. Se actualiza el estado cuando se resuelve la promesa.
- Muestra un spinner de carga mientras loading es true.

- Una vez cargados los datos, extrae:
    - Tipos: pokemon.types.map(t => t.type.name).
    - Estadísticas: HP, ataque, defensa, velocidad mediante find en pokemon.stats.
    - Habilidad principal (no oculta): pokemon.abilities.find(a => !a.is_hidden)?.ability.name.
    - Experiencia base: pokemon.base_experience.
    - Imagen: prioriza pokemon.sprites.other['official-artwork'].front_default, fallback a sprites.front_default.
- Define un objeto cardStyle con variables CSS personalizadas (--type-color y --type-light) basadas en el tipo principal para aplicar colores dinámicos.
- Renderiza:
    - Cabecera con ID formateado (ej. #001) y nombre.
    - Contenedor de imagen con borde circular y color de tipo.
    - Badges de tipo con colores específicos (definidos en el mapa typeColors).
    - Barras de progreso para estadísticas, con valores numéricos superpuestos.
    - Sección extra con experiencia base y habilidad, con estilo de etiqueta amarilla.
- Los estilos en PokemonCard.css incluyen:
    - Variables CSS para colores de tipo.
    - Efectos hover que elevan la tarjeta y agrandan la imagen.
    - Barra superior de color variable que indica el tipo.

- Diseño de barras de estadísticas con gradientes.
- Responsividad para móviles y tablets.

5.4 PokemonList

Archivos: PokemonList.jsx, PokemonList.css

Propósito:
Componente contenedor que muestra una cuadrícula (grid) de tarjetas PokemonCard. Obtiene la lista de Pokémon del store de Redux y gestiona la visualización de estados de carga, error o resultados vacíos.

Implementación Técnica:

- Utiliza useSelector para acceder a list, loading y error del store.
- Renderizado condicional:
  - Si loading es true: muestra un mensaje de carga con clase loading.
  - Si error no es null: muestra el mensaje de error.
  - Si list.length === 0: muestra "No se encontraron Pokémon".
  - En caso contrario, mapea list para renderizar un PokemonCard por cada elemento, pasando name como prop y usando key={pokemon.name}.
- Los estilos en PokemonList.css definen un grid responsivo:
  - grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)) para adaptarse al ancho.
  - Media queries para ajustar columnas en diferentes tamaños de pantalla (3 en escritorio, 2 en tablets, 1 en móviles).
  - Padding y márgenes para centrado.

5.5 SearchBar

Archivos: SearchBar.jsx, SearchBar.css

Propósito:
Proporciona la funcionalidad de búsqueda por nombre y filtrado por tipo de Pokémon. Incluye un campo de texto, un selector desplegable de tipos y un botón para resetear los filtros. La búsqueda por nombre aplica un debounce para optimizar el rendimiento.

Implementación Técnica:

- Conectado a Redux para leer searchTerm y filterType del store, y para despachar acciones setSearchTerm, setFilterType y resetFilters.
- Define un array pokemonTypes con objetos que contienen value, label y color para cada tipo, incluyendo la opción "Todos los tipos" (all).
- Estado local:
  - localSearch: sincronizado con el input de texto.
  - localType: sincronizado con el selector de tipos.
- **Debounce en búsqueda:**
  - Un useEffect escucha cambios en localSearch. Configura un temporizador de 300ms que, al cumplirse, despacha setSearchTerm si el valor es diferente al del store.
  - Limpia el temporizador en cada nuevo cambio para evitar múltiples despachos.
- Otros useEffect mantienen sincronizados los estados locales con los valores del store (por si se resetean externamente).
- Manejadores:

    - handleTypeChange: actualiza estado local y despacha setFilterType inmediatamente.
    - handleReset: limpia estados locales y despacha resetFilters.
    - handleSubmit (en el formulario): evita comportamiento por defecto y despacha setSearchTerm con el valor actual (útil si el usuario presiona Enter).
- Renderizado:
    - Formulario con input y botón de búsqueda que incluye el icono CgPokemon.
    - Selector de tipos con fondo dinámico basado en el color del tipo seleccionado (usando style).
    - Botón "Resetear filtros".
- Estilos en SearchBar.css:
    - Diseño coherente con la temática retro: bordes negros, sombras duras, tipografía 'Press Start 2P'.
    - El selector tiene una flecha personalizada y colores de fondo variables.
    - Media queries para apilar elementos en móviles.

Buenas Prácticas de Desarrollo Frontend

Arquitectura Basada en Componentes Funcionales

- **Componentes puros y reutilizables**: Cada pieza de la interfaz (Header, SearchBar, PokemonCard, etc.) está encapsulada en un componente independiente, lo que facilita su mantenimiento, prueba y reutilización en otras partes de la aplicación.
- **Separación de responsabilidades**: Los componentes se enfocan únicamente en la presentación y la lógica de interacción básica, delegando la gestión de datos y estado a Redux y a los servicios externos.
- **Uso de hooks**: Se emplean hooks nativos de React (useState, useEffect) y hooks personalizados (usePokemonList.js) para encapsular lógica compleja y mejorar la legibilidad del código.

Gestión de Estado con Redux Toolkit

- **Estado global centralizado**: El store de Redux contiene toda la información compartida (lista de Pokémon, página actual, filtros, estado de carga, errores), evitando la duplicación de datos y simplificando la sincronización entre componentes.
- **Inmutabilidad y predictibilidad**: Redux Toolkit garantiza la inmutabilidad del estado mediante Immer, y las acciones describen de forma clara los cambios que se producen.
- **Thunks asíncronos**: Las operaciones que involucran llamadas a la API (obtener lista de Pokémon, filtrar por tipo) se manejan mediante createAsyncThunk, separando la lógica asíncrona de los componentes y permitiendo un manejo uniforme de estados de carga y error

Consumo de API y Manejo de Datos

- **Capa de servicios**: Todas las llamadas HTTP se centralizan en pokemonService.js, utilizando Axios como cliente. Esto facilita la gestión de errores, la configuración de interceptores y la reutilización de peticiones.
- **Debounce en búsqueda**: Para evitar peticiones innecesarias a la API mientras el usuario escribe, se implementa un debounce de 300ms que retrasa el despacho de la acción setSearchTerm hasta que el usuario deja de escribir. Esto mejora el rendimiento y la experiencia de usuario.
- **Paginación eficiente**: La API se consulta con los parámetros offset y limit para obtener solo los 6 Pokémon necesarios por página, reduciendo la carga de datos y el tiempo de respuesta. Cuando se aplican filtros, se obtiene una lista completa (hasta 1000) y se página localmente, lo que es un compromiso aceptable dado que la PokéAPI no ofrece búsqueda por nombre en el servidor.

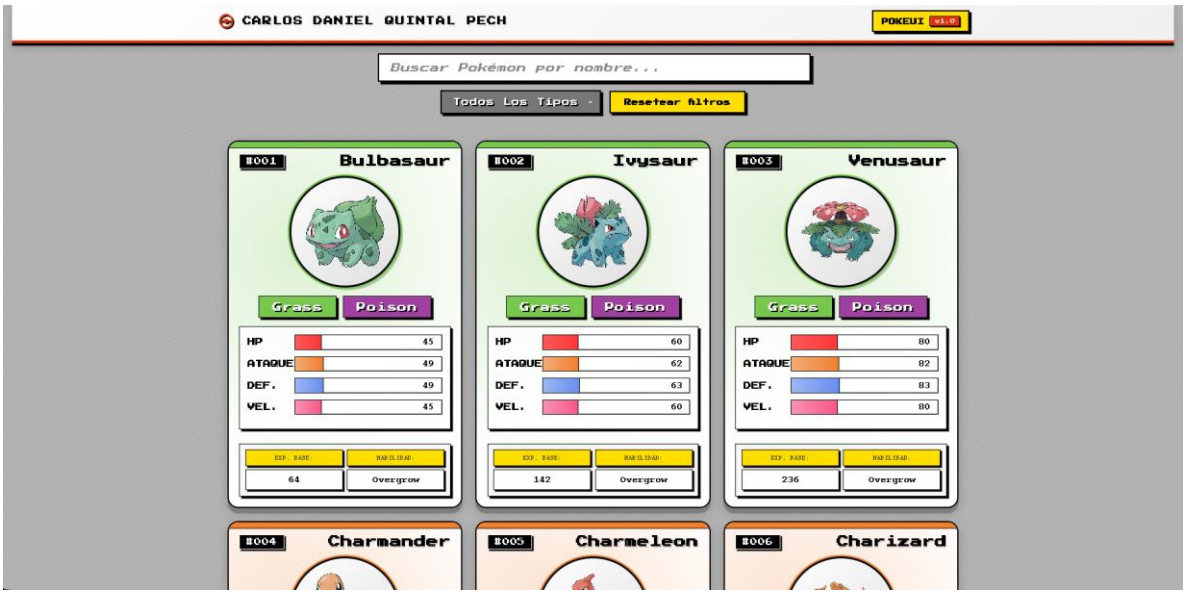
Manejo de Errores y Estados de Carga

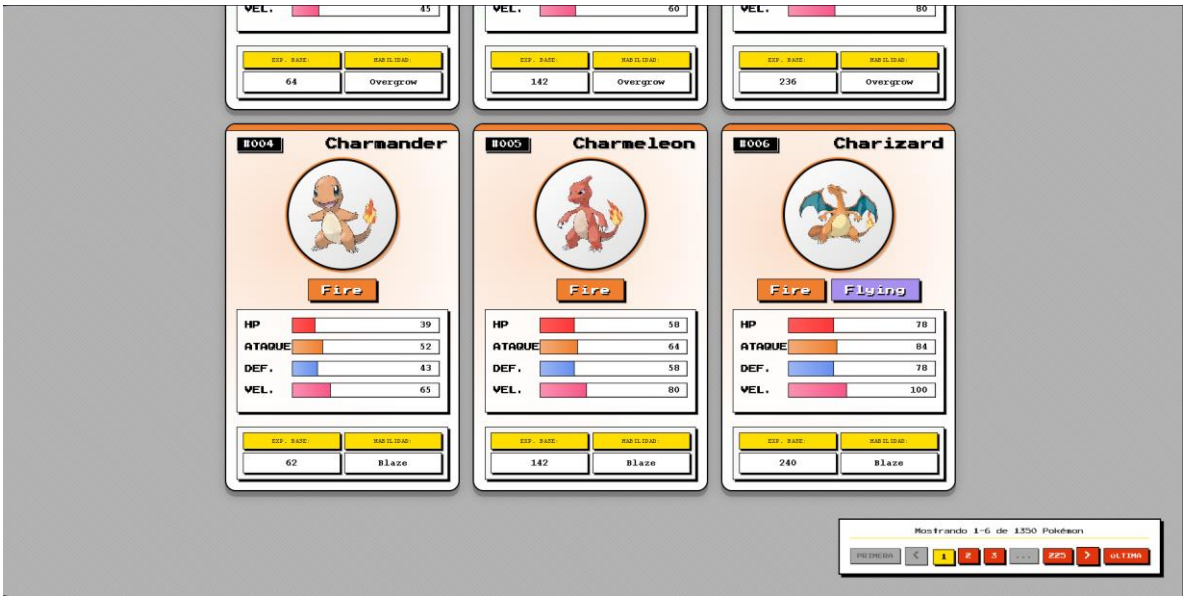
- **Feedback visual inmediato**: Los componentes PokemonList y PokemonCard muestran indicadores de carga (spinners o textos) mientras se espera la respuesta de la API, y mensajes de error claros si la petición falla.
- **Estados vacíos**: Cuando la búsqueda o el filtro no arrojan resultados, se muestra un mensaje "No se encontraron Pokémon", evitando que el usuario se encuentre con una pantalla en blanco.
- **Gestión de errores global**: El slice de Redux captura cualquier error en las peticiones y lo almacena en el estado, permitiendo que cualquier componente conectado pueda mostrarlo de forma consistente.

Diseño de Interfaz de Usuario (UI)

Consistencia Visual y Temática Retro

- **Paleta de colores**: Se utiliza una combinación de rojo (#e3350d), amarillo (#ffde00), negro y blanco, que evoca la estética de la Pokédex clásica. Los colores de los tipos de Pokémon se aplican en badges, barras de estado y bordes, proporcionando una identificación visual rápida.
- **Tipografía**: Se emplea la fuente 'Press Start 2P' para títulos y etiquetas principales, y 'Courier New' para textos largos y valores numéricos, logrando un aspecto retro pero legible.
- **Sombras y bordes**: Los componentes utilizan bordes negros gruesos y sombras duras (box-shadow con desplazamiento) para simular un efecto de relieve, característico de las interfaces de videojuegos clásicos.
- **Microinteracciones**: Efectos hover en tarjetas y botones (elevación, cambio de color, rotación de iconos) mejoran la sensación de interactividad y respuesta.



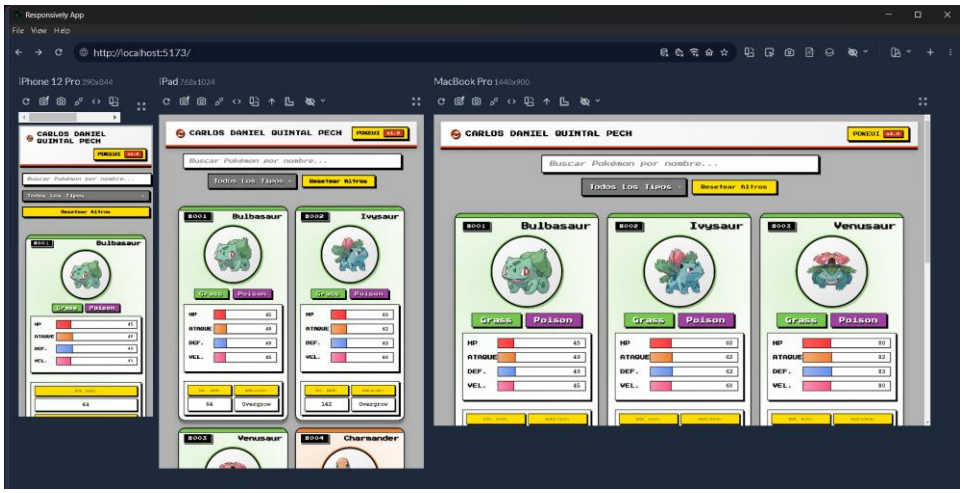


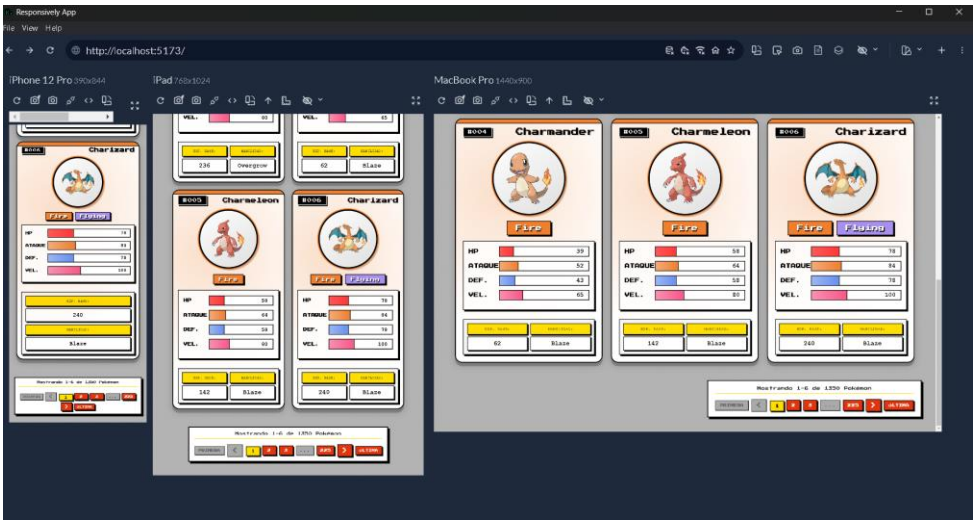
Diseño Responsive

- **Enfoque mobile-first**: Se han definido media queries para adaptar el layout a diferentes tamaños de pantalla:
    - **Móviles (≤600px)**: 1 columna en la lista de Pokémon, elementos apilados verticalmente, padding reducido.
    - **Tablets (601px-1024px)**: 2 columnas en la lista, ajustes de fuente y espaciado.
    - **Escritorio (≥1025px)**: 3 columnas, máximo aprovechamiento del espacio.
- **Unidades relativas**: Se utilizan rem, % y vh en lugar de píxeles fijos para garantizar la escalabilidad en diferentes resoluciones.
- **Contenedores flexibles**: La cuadrícula de Pokémon se basa en grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)), lo que permite que las tarjetas se adapten al ancho disponible sin necesidad de media queries adicionales.

Accesibilidad

- **Contraste adecuado**: Los textos sobre fondos de color mantienen un contraste suficiente (por ejemplo, texto blanco sobre rojo o negro, texto negro sobre amarillo) para garantizar la legibilidad.
- **Etiquetas semánticas**: Se emplean elementos HTML apropiados (header, main, form, button) y atributos alt en las imágenes, mejorando la navegación para lectores de pantalla.
- **Foco visible**: Los elementos interactivos (botones, inputs) muestran un estado de foco claramente diferenciado (por ejemplo, :focus-visible con borde adicional), facilitando la navegación por teclado.





Experiencia de Usuario (UX)

Navegación Intuitiva

- **Paginación clara**: Además de los botones "Anterior" y "Siguiente", se incluyen botones para ir a la primera y última página, así como números de página (limitados a 5) con la página actual resaltada. Un indicador "Mostrando X-Y de Z Pokémon" proporciona contexto sobre la posición actual.
- **Filtro automático por tipo**: Al seleccionar un tipo en el desplegable, la lista se actualiza inmediatamente, sin necesidad de presionar un botón adicional. Esto acelera la exploración y reduce pasos innecesarios.
- **Búsqueda con debounce**: La búsqueda por nombre se activa automáticamente después de una pausa en la escritura, ofreciendo resultados en tiempo real sin saturar la API.
- **Botón de reseteo**: Un botón "Resetear filtros" permite volver rápidamente al estado inicial (todos los tipos, sin búsqueda), mejorando la usabilidad cuando se aplican múltiples filtros.

Feedback Continuo

- **Indicadores de carga**: Mientras se obtienen los datos, se muestra un spinner o un mensaje de carga, evitando la incertidumbre del usuario.
- **Mensajes de error**: Si ocurre un error en la petición, se muestra un mensaje claro y se permite al usuario intentar de nuevo (mediante la recarga natural de la página o la interacción con los filtros).
- **Confirmación visual**: Los botones y elementos interactivos cambian de apariencia al pasar el mouse o al hacer clic, confirmando que la acción ha sido registrada.

Optimización de Tareas Frecuentes

- **Paginación con scroll suave**: Al cambiar de página, la ventana se desplaza suavemente hacia arriba, evitando que el usuario tenga que hacerlo manualmente y manteniendo el contexto.
- **Persistencia de filtros**: Los filtros aplicados (búsqueda y tipo) se mantienen al navegar entre páginas, lo que permite explorar resultados filtrados sin tener que reingresar los criterios.
- **Información relevante en tarjetas**: Cada tarjeta muestra datos clave (estadísticas, experiencia, habilidad) de forma compacta pero legible, permitiendo al usuario tomar decisiones rápidas.

Diseño Centrado en el Usuario

- **Jerarquía visual**: Los elementos más importantes (imagen del Pokémon, nombre, tipos) tienen mayor peso visual, mientras que los detalles secundarios (estadísticas, experiencia) se presentan de forma ordenada pero menos prominente.
- **Uso de iconos**: El icono de Pokébola en el botón de búsqueda y en el header refuerza la identidad temática sin añadir complejidad.
- **Espaciado y legibilidad**: Se ha cuidado el padding y los márgenes para que los elementos no se sientan apretados, incluso en pantallas pequeñas.